

Министерство образования Республики Беларусь
Учреждение образования
«Брестский Государственный технический университет»
Кафедра ИИТ

Лабораторная работа №1
По дисциплине «Мобильные приложения»
Тема: «Дизайн и события мобильных приложений»

Выполнил:
Студент 3 курса
Группы АС-66
Цеван К.А.
Проверил:
Козинский А. А.

Цель работы: разработка android-приложения “Память”

Ход работы

Лабораторная работа №1. Игра “Память” на Android

Необходимо реализовать интерфейс приложения для последовательного открытия пар карточек для запоминания. Приложение в минимальном представлении должно:

отображать игровое поле минимального размера 4x4

в каждый момент времени на экране отображается не более одной картинки. При открытии каждой следующей картинки происходит парное удаление с игрового поля только в случае совпадения текущей и предыдущей картинки.

предоставлять возможность перезапустить игру

Бонусы (то, что способствует оценке выше 4)

– Возможность настраивать установки игрового поля (переменный размер игрового поля, цвета игры, наборы картинок и т.п.)

– Возможность присутствия на игровом поле не только ПАР, но и ТРОЕК картинок

– Запись информации (пользователи, рекорды, время игры и т.п.)

– Собственное оформление игры (отображение информации для пользователя время, число оставшихся пар-троек, рекорды, мотивационные тексты и т.п.)

Функциональные требования

1. Основной функционал:

- Игровое поле размером 4×4 (минимально)
- В каждый момент времени открыта только одна карточка
- Карточка открывается на 0.5 секунды, затем автоматически закрывается
- Во время открытия карточки другие карточки неактивны
- При совпадении текущей и предыдущей карточки - обе удаляются с поля
- Возможность перезапуска игры

2. Дополнительный функционал (бонусы):

- Настройка размера игрового поля (от 2×2 до 6×6)
- Режим игры "тройки" (3 одинаковые карточки)
- Запись рекордов в базу данных
- Таймер и счетчик ходов

- Мотивационные сообщения при завершении игры

3. Требования к интерфейсу:

- Наличие признаков авторства во всех окнах
- Интуитивно понятный дизайн
- Приятная цветовая гамма

Графическое изображение интерфейса

MainActivity:

```
package com.example.memorygame

import android.content.Context
import android.os.Bundle
import android.os.Handler
import android.os.Looper
import android.widget.Button
import android.widget.TextView
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity
import androidx.recyclerview.widget.GridLayoutManager
import androidx.recyclerview.widget.RecyclerView

class MainActivity : AppCompatActivity() {

    private lateinit var rvBoard: RecyclerView
    private lateinit var tvPairs: TextView
    private lateinit var tvTime: TextView
    private lateinit var tvBest: TextView
    private lateinit var tvMoves: TextView
    private lateinit var btnRestart: Button

    private lateinit var adapter: MemoryAdapter
    private lateinit var cards: MutableList<MemoryCard>

    private var indexOfSingleSelectedCard: Int? = null
    private var moves = 0

    private var secondsElapsed = 0
    private var startTimeMs: Long = 0L

    private val handler = Handler(Looper.getMainLooper())
    private val timerRunnable = object : Runnable {
        override fun run() {
            secondsElapsed++
            updateTimeUI()
            handler.postDelayed(this, 1000)
        }
    }

    private val prefs by lazy {
        getSharedPreferences(PREFS_NAME, Context.MODE_PRIVATE)
    }
```

```

}

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_main2)

    Toast.makeText(
        this,
        "Лабораторная работа №1.\nИгра «Память».\nВыполнил: Цеван  
Константин\nГруппа: АС-66",
        Toast.LENGTH_LONG
    ).show()

    rvBoard = findViewById(R.id.rvBoard)
    tvPairs = findViewById(R.id.tvPairs)
    tvTime = findViewById(R.id.tvTime)
    tvBest = findViewById(R.id.tvBest)
    tvMoves = findViewById(R.id.tvMoves)
    btnRestart = findViewById(R.id.btnRestart)

    rvBoard.layoutManager = GridLayoutManager(this, 4)

    btnRestart.setOnClickListener { setupBoard() }

    showBestRecord()
    setupBoard()
}

override fun onStop() {
    super.onStop()
    handler.removeCallbacks(timerRunnable)
}

private fun setupBoard() {
    stopAndResetTimer()
    startTimer()
    startTimeMs = System.currentTimeMillis()

    val images = listOf(
        R.drawable.baseline_cake_24,
        R.drawable.baseline_heart_broken_24,
        R.drawable.outline_boy_24,
        R.drawable.outline_brightness_1_24,
        R.drawable.outline_brightness_5_24,
        R.drawable.outline_brush_24,
        R.drawable.outline_call_24,
        R.drawable.ic_android_black_24dp
    )

    val fullList = (images + images).shuffled()
    cards = fullList.map { MemoryCard(it) }.toMutableList()

    indexOfSingleSelectedCard = null
    moves = 0
    updateMovesUI()

    adapter = MemoryAdapter(cards) { position -> onCardClicked(position) }
}

```

```

        rvBoard.adapter = adapter

        updatePairsUI()
        showBestRecord()
    }

    private fun onCardClicked(position: Int) {
        val clickedCard = cards[position]
        if (clickedCard.isMatched || clickedCard.isFaceUp) return

        moves++
        updateMovesUI()

        val prevIndex = indexOfSingleSelectedCard

        if (prevIndex == null) {
            clickedCard.isFaceUp = true
            indexOfSingleSelectedCard = position
            adapter.notifyItemChanged(position)
        } else {
            val prevCard = cards[prevIndex]
            clickedCard.isFaceUp = true

            if (prevCard.identifier == clickedCard.identifier) {
                prevCard.isMatched = true
                clickedCard.isMatched = true
                indexOfSingleSelectedCard = null
                adapter.notifyItemChanged(prevIndex)
                adapter.notifyItemChanged(position)
            } else {
                prevCard.isFaceUp = false
                indexOfSingleSelectedCard = position
                adapter.notifyItemChanged(prevIndex)
                adapter.notifyItemChanged(position)
            }
        }
    }

    updatePairsUI()

    if (pairsLeft() == 0) {
        handler.removeCallbacks(timerRunnable)
        val playedSeconds = ((System.currentTimeMillis() - startTimeMs) /
1000).toInt()
        trySaveBestRecord(playedSeconds)
        showBestRecord()
    }
}

private fun pairsLeft(): Int = cards.count { !it.isMatched } / 2

private fun updatePairsUI() {
    tvPairs.text = "Осталось пар: ${pairsLeft()}"
}

private fun updateMovesUI() {
    tvMoves.text = "Ходов: $moves"
}

```

```

private fun startTimer() {
    handler.removeCallbacks(timerRunnable)
    handler.postDelayed(timerRunnable, 1000)
}

private fun stopAndResetTimer() {
    handler.removeCallbacks(timerRunnable)
    secondsElapsed = 0
    updateTimeUI()
}

private fun updateTimeUI() {
    tvTime.text = "Время: ${formatTime(secondsElapsed)}"
}

private fun formatTime(totalSeconds: Int): String {
    val minutes = totalSeconds / 60
    val seconds = totalSeconds % 60
    return String.format("%02d:%02d", minutes, seconds)
}

private fun showBestRecord() {
    val best = prefs.getInt(KEY_BEST_SECONDS, -1)
    tvBest.text = if (best <= 0) "Рекорд: --:--" else "Рекорд:
${formatTime(best)}"
}

private fun trySaveBestRecord(currentSeconds: Int) {
    if (currentSeconds <= 0) return
    val best = prefs.getInt(KEY_BEST_SECONDS, -1)
    if (best <= 0 || currentSeconds < best) {
        prefs.edit().putInt(KEY_BEST_SECONDS, currentSeconds).apply()
    }
}

companion object {
    private const val PREFS_NAME = "memory_game_prefs"
    private const val KEY_BEST_SECONDS = "best_seconds"
}
}

```

Окно 2: Главное меню



Вывод: В ходе выполнения лабораторной работы было разработано мобильное приложение "Игра Память" для Android.